Storage is separate to Identity itself. You are free to choose your own storage mechanism and create a custom provider to work with it if one doesn't already exist. The default implementation of Identity storage relies on the `Microsoft.AspNetCore .Identity.EntityFrameworkCore` NuGet package, which works with relational databases. You add that to the project using `dotnet add package` if you are using the CLI, or `install-package` for the Package Manager Console, by adding a reference to the project file directly or using the Visual Studio tool for managing NuGet packages in the solution. The Identity UI is in a package called `Microsoft.AspNetCore .Identity.UI`, so that needs to be added too. Once both packages have been added, ensure that `dotnet restore` is executed.

9.2.1 Creating a user

The next step requires you to create a class to represent your user. A key feature of this class is that it must derive from `IdentityUser` if it is to work with the Identity framework. The `IdentityUser` type has several properties already defined, including `UserName`, `Email`, `PhoneNumber,` and some authentication workflow-specific properties, such as `LockoutEnabled`. This indicates whether the lockout feature has been enabled, whereby accounts are automatically locked after a specified number of invalid login attempts. You would enable this to prevent *brute force attacks*—usually automated attempts to log in by a hacker systematically trying all possible password combinations.

The `IdentityUser` also includes a property named `HashedPassword`. This provides a clue to the fact that Identity hashes passwords using a cryptographically secure algorithm before storing them. Passwords should never, ever, *ever* be stored as plain text. Users are lazy and tend to reuse passwords. Chances are they may use the same one in your application as they use for their bank. If a hacker was able to obtain a copy of your users' data, you could be guilty of providing them with access to all sorts of resources outside your